Spatial Pooling Graph Convolutional Network for Hyperspectral Image Classification

Xiangrong Zhang, *Senior Member, IEEE*, Shutong Chen, Peng Zhu, Xu Tang, *Senior Member, IEEE*, Jie Feng, *Member, IEEE*, and Licheng Jiao, *Fellow, IEEE*

***Abstract*— Graph convolution networks (GCNs) have been applied in a variety of fields due to their powerful ability in processing graph-like data. However, the massive number of hyperspectral pixels makes it challenging to define general graph structures on hyperspectral images (HSIs). On the other hand, convolutional neural networks (CNNs) take in regular image regions with fixed square size, and have demonstrated impressive accuracy while being efficient in computation. Inspired by the classification framework of CNNs, we develop a GCN-based model that generates effective local spectral–spatial features for HSI classification. Specifically, graph convolutions are performed separately on every local region, which significantly limits the graph's size. While graph convolution extracts features of every pixel, it does not reduce the number of them. To fuse suitable representations for the classification task, we develop a graph pooling operation to preserve classification-specific features and reduce redundant pixels. Based on local regions of HSIs, pooling in the graph domain is equivalent to spatial pooling in the spatial domain. The proposed method is thus named the spatial pooling graph convolutional network (SPGCN). Experimental results on several typical datasets demonstrated that the proposed SPGCN provides competitive results compared with other state-of-the-art CNN-based methods.**

***Index Terms*— Graph convolutional network (GCN), graph pooling, hyperspectral image (HSI) classification, spectral–spatial feature extraction.**

I. Introduction

RECENT years with the rapid development of remote sensing techniques, it is easier to obtain more and more hyperspectral data [1]. Compared with general remote sensing images, the spectral resolution of hyperspectral images (HSIs) reaches the nanometer level, which often results in hundreds of spectral bands. Therefore, HSIs have rich spectral–spatial information that enables them with a wide range of applications in many fields, such as agriculture [2], environment [3], ubran [4], and military [5]. The goal of HSI classification is to assign each hyperspectral pixel with a proper land-cover label [6]. In particular, the high dimension of spectral bands makes it possible to discriminate various materials with high accuracy.

A variety of traditional methods have been applied to HSI classification. Some of the learning methods have a wide range of applications, including support vector machines (SVMs) [7], k-nearest neighbors (kNNs) [8], random forest [9], and sparse representations [10]. These basic machine learning methods are mainly handcrafted, which are initially designed to solve the general classification problem. In addition to the above learning methods, some handcraft feature extraction methods have also been successfully applied. For example, principal component analysis (PCA) [11] and linear discriminant analysis (LDA) [12] are used to map and transform high-dimensional spectral space to low-dimensional feature space. 3-D wavelet analysis (WT) [13] and Markov random field (MRF)-based models [14] can extract spectral–spatial features, which have achieved wide applications. Though being fast in speed, these shallow methods often perform unsatisfactory results due to high-correlation and high-dimensional characteristics of hyperspectral data. To handle the complex information of HSIs more thoroughly and effectively, we need models with better representative capabilities.

With the rapid development of deep learning, classification performance for HSI has gone up to a new level. Compared with shallow learning methods, deep learning methods can extract more robust features and have achieved better classification performance by manipulating the input data from low level to high level. For example, stack autoencoder (SAE) [15], deep belief network (DBN) [16], and recurrent neural network (RNN) [17] have been adopted to extract deep features. While HSIs are 3-D cube-like data in the spatial and spectral domains, these early methods were applied only in the spectral domain. To make full use of both the spectral and spatial information, convolutional neural networks (CNNs) were applied due to their nature capability in processing cube-like data. A small patch of HSI instead of a 1-D feature is fed into CNNs. At the beginning, there were 2-D CNN and 3-D CNN [18] with a few layers. Further, deeper models with residential blocks were developed [19], [20]. Attention techniques apply different weights to different feature dimensions, and have been adopted to CNNs in several methods recently [21]–[24]. Furthermore, there have been diverse methods based on different deep learning models [25]. Zhang and Gao [26] used an extended MRF model to combine spatial and multiple semantic information. Zhu *et al.* [27] applied capsule networks to simplify CNNs. In [28], recursive autoencoders (RAEs) were used to develop unsupervised feature representations. Although CNN-based methods have achieved significant performance, there is still a drawback. To be specific, conventional CNN models do not consider the relationship between pixels.

Manuscript received June 10, 2021; revised September 23, 2021; accepted December 1, 2021. Date of publication January 4, 2022; date of current version March 8, 2022. *(Corresponding author: Xiangrong Zhang.)*

The authors are with the Key Laboratory of Intelligent Perception and Image Understanding of Ministry of Education, Xidian University, Xi'an 710071, China (e-mail: xrzhang@mail.xidian.edu.cn).

Digital Object Identifier 10.1109/TGRS.2022.3140353

1558-0644 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 15:07:25 UTC from IEEE Xplore. Restrictions apply.

Although attention-based CNNs attach different weights to different features, the attention masks are often multiplied to the features based on dot product, which is not a direct measurement of the relationship between pixels. Therefore, building graphs can contribute to overcoming this issue. Edges naturally represent the relationship among vertices, and graph convolutional networks (GCNs) can extract information based on the graph structures.

Many real-life data are in non-Euclidean space, and can be considered as graphs composed of points connected by lines. Such kinds of data are difficult to be processed by conventional grid convolution techniques, so recently, GCNs are introduced to handle the issue. The typical GCN was proposed by Kipf and Welling [29], which can naturally propagate vertex features based on graph structures. Effective as it is, the method is not flexible enough. During graph convolution, all data need to participate in the construction of the graph, which will be costly in computation when the size of the data is large. Also, the scale of the graph is fixed, so the only way to predict new samples is to retrain the graph. There are mainly two ways to overcome the above issues: subgraph training and graph pooling. Subgraph training trains networks on a set of local subgraphs. Graphsage [30] fixed the size of GCN, and trained on the local neighborhoods of each vertex. Cluster-GCN [31] performed graph cluster methods to cluster the graph, randomly selected small subgraphs and trained on them separately. On the other hand, graph pooling pools the graph into smaller graphs. Defferrard *et al.* [32] applied cluster methods to combine the vertex pairs, which resulted in a binary tree. Ying *et al.* [33] proposed a learnable matrix to reduce the adjacent matrix's size and the vertices. In this article, the purpose of graph construction is to serve the classification of the center pixel. Therefore, all the vertices in the graph should be relative to the center vertex, and graph pooling methods are required to fuse classification information.

The recent GCN methods for HSI classification can be roughly divided into three categories: image-based, superpixel-based, and selection-based. For the image-based methods, each pixel represents one vertex and graphs are constructed over whole pixels of the HSI. Therefore, one forward propagation can predict all pixels. These methods are straightforward but tend to suffer from high computation costs when the HSI is large. Mou *et al.* [34] constructed a graph over all pixels of the HSI, performed graph convolution, and finally predicted the pixels at one time. Bai *et al.* [35] used the residential connection to form deep GCN and dynamically update the graph structures. The superpixel-based methods were initially proposed by Wan *et al.* [36], and the core idea is to pre-segment the HSI into small areas to alleviate the high memory cost. Since pixels in one superpixel have similar features, they can be treated as one vertex. There have been several works recently [37]–[39]. The selection-based methods select a subset of pixels and construct a graph over them instead of the entire HSI. Some feature extraction techniques such as CNN are first implied on every pixel to extract features, and GCN is performed over these features. Hong *et al.* [40] randomly selected pixels from arbitrary regions of the HSI and utilized a GCN over these pixels. He *et al.* [41] used dual GCN to integrate label and vertex information separately and mixed them together. While the above methods use GCN to integrate features far in spatial distance, our method extracts features only on local regions.

In this article, we propose a new method called spatial pooling GCN (SPGCN). We present a pure GCN spectral–spatial feature extraction technique to extract local region features, which is mainly composed of two parts: graph convolution and spatial pooling. As graph convolution is a powerful tool to fuse different vertex features based on their graph relationship, we treat local patch pixels as vertices and perform graph convolution on them. While graph convolution effectively propagates information between pixels, it does not extract any global feature. In addition, some pixels that are spatially away from the center pixel may contribute little to the classification result. To preserve essential features and reduce redundant pixels, we propose a spatial pooling method to reduce the spatial size of the HSI patch. After the pooling operation, the size of the graph and the number of pixels are reduced. Since shallow models cannot extract deep features, the proposed SPGCN is constructed by stacking several spatial pooing graph convolution (SPGC) modules. Therefore, spatial sizes are gradually reduced by several spatial pooling layers. Finally, a fully connected layer followed by a softmax layer is applied to obtain the classification results. While current GCN-based methods are built on global structures of the HSI and predict a number of pixels each time, our method is built on local regions and predicts one pixel each time, which is similar to CNN-based methods.

We demonstrate that graph convolution is an effective tool to extract spectral–spatial features, and our method performs competitively compared with CNN-based methods.

The main contributions are as follows:

1) We propose a SPGCN for HSI classification. The model is mainly constructed by stacking several SPGC modules, which fuses local spectral–spatial features based on graph convolution and reduces redundant pixels by graph pooling.
2) We apply a spatial pooling operation that effectively preserves important features of graphs and reduces the spatial size of HSI patches.
3) The proposed method achieves competitive performance on three typical datasets, demonstrating that graph convolution is an effective tool to extract spectral–spatial features.

The remaining parts of this article are organized as follows. Section II reviews the work in graph convolution and graph pooling. Section III introduced the proposed SPGCN model. Experiment analysis of SPGCN is discussed in Section IV. Finally, the conclusion and future research direction are shown in Section V.

II. Related Work

A. Graph Convolution Network (GCN)

In the following, we review the graph convolution architecture. Let $\mathbf{G} = (\mathbf{V}, \mathbf{E})$ be an undirected graph, where $\mathbf{V}$ is

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 15:07:25 UTC from IEEE Xplore. Restrictions apply.

the vertex set of size n and $\mathbf{E} \in \mathbb{R}^{n\times n}$ represents the edge set. e_{ij} denotes the edge $(v_i, v_j) \in \mathbf{E}$. For each vertex v_i in the graph, there is a feature vector $x_i \in \mathbb{R}^d$ associated with it, where d is the dimension of the feature vector. The whole feature matrix $[x_1, x_2, \ldots, x_n]^T$ can be represented as $\mathbf{X} \in \mathbb{R}^{n\times d}$. Considering the architecture made by Kipf and Welling [29], graph convolution operation can be written as

$$\mathbf{H}^{(l+1)} = \sigma\left(\widetilde{\mathbf{A}}\mathbf{H}^{(l)}\mathbf{W}^{(l)}\right) \tag{1}$$

where $\mathbf{H}^{(l)} \in \mathbb{R}^{n\times d}$ is the feature matrix computed after l steps. For $l = 0$, $\mathbf{H}^0 = \mathbf{X}$. $\mathbf{W}^{(l)}$ is the trainable matrix of the lth layer. σ is the activation function, typically the element-wise rectified linear unit (ReLU). The matrix $\widetilde{\mathbf{A}}$ is derived from the observed graph $\mathbf{G}$. A commonly applied definition of $\widetilde{\mathbf{A}}$ is denoted as

$$\widetilde{\mathbf{A}} = \mathbf{I} + \mathbf{D}^{-\frac{1}{2}}\mathbf{A}\mathbf{D}^{-\frac{1}{2}} \tag{2}$$

where $\mathbf{A}$ is the adjacency matrix based on the edge set $\mathbf{E}$. $\mathbf{D}$ is a diagonal matrix where $\mathbf{D}_{ii} = \sum_j \mathbf{A}_{ij}$. $\mathbf{I}$ is the identity matrix. After L layers of graph convolution operation, the final output is denoted as $\mathbf{Z} = \mathbf{H}^{(L)}$. Let $\mathbf{Y} = [y_1, y_2, \ldots, y_n]^T$ be the label matrix of every vertex. For a training set of size K, $\{(\mathbf{x}_i, \mathbf{y}_i)|i = 1, 2, \ldots, K\}$, $\mathbf{W}^{(l)}$ is learned by backpropagating the loss between $\mathbf{Z}$ and $\mathbf{Y}$. Apart from the above GCN architecture, there are a large number of approaches to aggregate graph features. Schlichtkrull *et al.* [42] introduced relational GCN to deal with knowledge graphs. Wu *et al.* [43] simplified GCN to one layer and replaced the normalized adjacency matrix $\widetilde{\mathbf{A}}$ with the kth power of $\widetilde{\mathbf{A}}$, which is $\widetilde{\mathbf{A}}^k$. Hamilton *et al.* [30] fixed the size of the network and performed GCN locally on every vertex. These methods provide different aspects of graph convolution.

B. Graph Pooling

Pooling is a systematic approach in CNNs. The pooling layer performs a nonlinear operation to reduce features' size, thereby reducing the number of learnable parameters and the computational cost. As a result, pooling can prevent overfitting, and allows a deeper network. The application of pooling operation on graphs is similar to pooling on grid-like data in many ways. When the purpose is to obtain the graph's overall feature, graph pooling can significantly reduce the size of the graph and preserve main features. Fig. 1 gives an illustration of graph pooling. The formal graph pooling is defined as follows:

$$\mathbf{E}_p = P_E(\mathbf{E}) \tag{3}$$

$$\mathbf{V}_p = P_V(\mathbf{V}) \tag{4}$$

where P_E is the edge pooling and P_V is the vertex pooling. In this way, both the edge and vertex numbers are reduced, resulting in a smaller graph. Since there is no explicit contextual relationship in graph structures, new pooling strategies are proposed to overcome this issue. Zhang *et al.* [44] sorted vertices according to their structural roles within the graph. In [33], Ying *et al.* proposed a learnable matrix to pool the edges and vertices. The matrix is differentiable, which is different from max pooling and average pooling. Graph u-nets [45] applies a trainable projection vector and performs vertex selection. Based on the 1-D footprint of each vertex, top-k indices are selected.

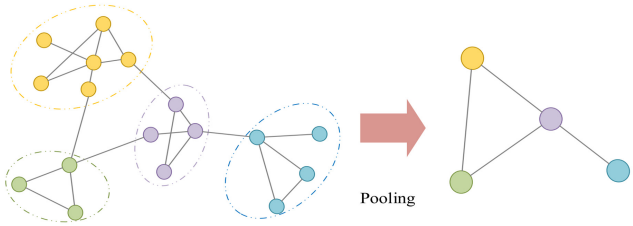


Fig. 1. Illustration of graph pooling methods. Different colors denote vertices of different sets. After graph pooling, vertices with the same color are pooled into one vertex.

III. Methodology

In this section, we detail the proposed SPGCN. Let $\mathbf{X} \in \mathbb{R}^{h\times w\times c}$ be the HSI, where h and w denote the height and width of the spatial dimension, and c denotes the number of bands of spectral dimension. Similar to CNN, the sample set of our GCN-based method consists of fixed length square cubes, where the center pixels of the square cubes remain to be classified. Therefore, a total number of $h \times w$ square cubes are cropped. Let $\mathscr{X} = \{\mathbf{X}_1, \mathbf{X}_2, \ldots, \mathbf{X}_N\}$ be the set of square cubes, where $\mathbf{X}_i \in \mathbb{R}^{s\times s\times c}$, $N = h \times w$. s is the spatial size of the square cube. Their corresponding land-cover label set is denoted as $\mathscr{Y} = \{\mathbf{Y}_1, \mathbf{Y}_2, \ldots, \mathbf{Y}_N\}$, where $\mathbf{Y}_i \in \mathbb{R}^d$. d is the number of classes. Then, the sample set can be represented as $(\mathscr{X}, \mathscr{Y})$.

The overall structure is shown in Fig. 2. The commonly used CNN kernel is replaced by graph convolution to extract spectral–spatial features. Graphs are built over pixels and only GCN instead of CNN is performed to extract local features. Furthermore, graph pooling is applied to reduce redundant pixels and preserve essential features.

The network is mainly composed of several SPGC modules. Note that this SPGC module can be further stacked with common CNN structures easily. The module consists of two parts: graph convolution and spatial pooling. We first treat pixels as graph vertices and perform graph convolution on them. Graph convolution provides a powerful tool to fuse local features based on their graph relationships. After graph convolution, the features of each pixel is the weighted sum of all pixel features. Since spatially closer pixels are more likely to be similar, pixels that are spatially away from the center pixel can be removed during convolution. Therefore, a graph pooling layer based on the spatial relationship is applied to reduce the graph's size and the vertices' size. Finally, a fully connected layer with a softmax activation function is applied to the smallest patch to obtain the classification results.

A. Spectral–Spatial Graph Convolution

Graph convolution can fuse each vertex as a weighted sum of the neighbor vertices, thereby effectively extract vertex features based on their graph relationships. However, the computational cost of GCNs becomes high when the size

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 15:07:25 UTC from IEEE Xplore. Restrictions apply.

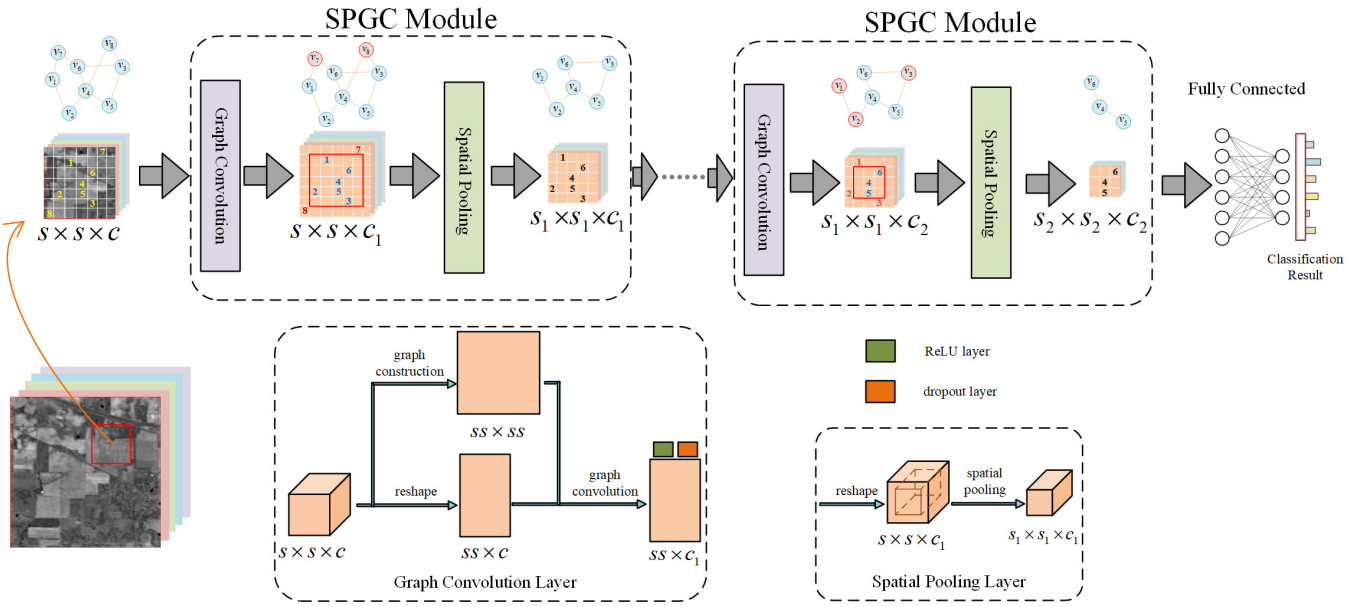


Fig. 2. Overview of the proposed SPGCN. The architecture is stacked of several SPGC modules and a fully connected layer. The spatial size of the HSI patch is reduced after each SPGC module. Some pixels $v_1, v_2, \ldots, v_8$ are marked to exhibit the function of the module. After the pixel-based graph convolution, pixels outside the red subwindow are removed, which corresponds to the red vertices v_7 and v_8. In another way, the spatial-based graph pooling layer preserved the blue vertices $v_1, v_2, \ldots, v_6$, which are inside the red box, result in a smaller graph.

of the graph increases. To overcome this issue, we intend to build graphs on local regions instead of constructing a complete graph of the HSI. Specifically, for a HSI square cube $\mathbf{X}_i \in \mathbb{R}^{s\times s\times c}$, we consider each pixel $x_j \in \mathbb{R}^c$ in $\mathbf{X}_i$ as the vertex feature. We further let v_j be the vertex of x_j. Then, the number of vertices in a square cube is $n = s \times s$, and $\mathbf{V}_i = \{v_1, v_2, \ldots, v_n\}$ denotes the vertex set of the square cube X_i. Furthermore, the set of vertices of the HSI is denoted as $\mathscr{V} = \{\mathbf{V}_1, \mathbf{V}_2, \ldots, \mathbf{V}_N\}$, where $N = h \times w$ is the number of pixels in the HSI. We further construct edges based on the vertices set $\mathscr{V}$. A variety of methods have been proposed for the construction of $\widetilde{\mathbf{A}}$. Graph attention networks (GAT) [46] applied multihead attention technique to learn edges from the vertex features of the current layers. Zhang *et al.* [47] adopted a Bayesian approach, viewing the observed graph as a realization from a parametric family of random graphs. While these methods may contribute to learning a better graph, in this article, we simply construct graphs based on the Gaussian similarity to better show the basic performance of the SPGCN. Specifically, For a vertex set $\mathbf{V}_i$ of an HSI square cube $\mathbf{X}_i$, adjacent matrix $\mathbf{A}_i$ can be calculated as

$$(\mathbf{A}_i)_{ab} = \exp\left(-\frac{\|x_a - x_b\|^2}{2\gamma}\right), \quad a, b = 1, 2, \ldots, n \tag{5}$$

where γ is the scale parameter of the Gaussian kernel and exp is the natural exponential funtion. Therefore, $\mathscr{A} = \{\mathbf{A}_1, \mathbf{A}_2, \ldots, \mathbf{A}_N\}$ is the adjacent matrix set. Combining $\mathscr{A}$ and $\mathscr{V}$, the graph set of the HSI is denoted as $\mathscr{G} = \{\mathbf{G}_1, \mathbf{G}_2, \ldots, \mathbf{G}_N\}$, where $\mathbf{G}_i = \{\mathbf{V}_i, \mathbf{A}_i\}$.

The above method construct graphs based on every local HSI patch and obtain the graph set $\mathscr{G} = \{\mathbf{G}_i = (\mathbf{V}_i, \mathbf{A}_i) | i = 1, 2, \ldots, N\}$. To extract local spectral–spatial features and reduce the computational cost, we apply GCNs separately on each HSI local patch $\mathbf{X}_i \in \mathbb{R}^{s\times s\times c}$ from the patch set $\mathscr{X} = \{\mathbf{X}_1, \mathbf{X}_2, \ldots, \mathbf{X}_N\}$. The idea is inspired by the widely adopted CNN-based classification framework, which extracts features on local regions of HSI. Furthermore, previous GCN-based architecture also has the same idea. In [30], a fixed-scale GCN is proposed, and graph convolution is performed separately on every local region of the graph.

Therefore, for every adjacent matrix in set $\mathscr{A} = \{\mathbf{A}_1, \mathbf{A}_2, \ldots, \mathbf{A}_N\}$ and every HSI patch in set $\mathscr{X} = \{\mathbf{X}_1, \mathbf{X}_2, \ldots, \mathbf{X}_N\}$, we omit the subscipt in $\mathbf{A}_i$, $\mathbf{X}_i$ for simplicity, and graph convolution is defined as follows:

$$\mathbf{H}^{(l')} = \sigma\left(\widetilde{\mathbf{A}}^{(l)}\mathbf{H}^{(l)}\mathbf{W}^{(l)}\right) \tag{6}$$

$$\widetilde{\mathbf{A}}^{(l)} = \mathbf{I} + \mathbf{D}^{(l)^{-1}}\mathbf{A}^{(l)} \tag{7}$$

where l is the number of layers. $\mathbf{D}^{(l)}$ is the degree matrix of $\mathbf{A}^{(l)}$, $\mathbf{D}^{(l)}_{aa} = \sum_b \mathbf{A}^{(l)}_{ab}$. $\mathbf{W}^{(l)}$ is the learnable parameter, and is shared among different HSI patches. σ is the activation function, for which we use the ReLU. The graph convolution provides a local feature fusion technique, and each pixel aggregates every pixel's feature in $\mathbf{H}^{(l)}$ based on the graph relationship. For the first layer, $\mathbf{H}^{(0)} = \mathbf{X}$, $\mathbf{A}^{(0)} = \mathbf{A}$. Note that the adjacent matrix $\mathbf{A}^{(l)}$ varies with the number of layers, and $\mathbf{H}^{(l')}$ is not the same as $\mathbf{H}^{(l+1)}$. Here, $\mathbf{H}^{(l')}$ is the output of the GCN, and is an intermediate feature representation, which means that we need further operation on $\mathbf{H}^{(l')}$ to obtain the next layer's feature $\mathbf{H}^{(l+1)}$. In Section III-B, we will discuss the spatial pooling operation, which handles the above issues.

B. Spatial Pooling

Graph convolution provides an effective way to extract spectral–spatial features based on local regions. However, there still remain some issues to be solved. Global features are

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 15:07:25 UTC from IEEE Xplore. Restrictions apply.

essential for the classification of center pixels, but the above graph convolution process maintains every pixel. Furthermore, not all the pixels contribute to the classification, and this negative effect may be severe when the HSI patches are large in spatial size. Consequently, it is necessary to refine the feature representations $\mathbf{H}^{(l')}$, which is obtained from the graph convolution operation. In this article, we use pooling methods to accomplish the above challenges.

For the lth layer, the corresponding graph is denoted as $\mathbf{G}^{(l)} = \{\mathbf{V}^{(l)}, \mathbf{A}^{(l)}\}$. We perform a graph pooling operation to reduce the graph size. Therefore, $\mathbf{G}^{(l+1)}$ is the subgraph of $\mathbf{G}^{(l)}$, and is also the graph of the $(l+1)$th layer. The subgraph set of all graph convolution layers is denoted as $\mathbf{G} = \{\mathbf{G}^{(0)}, \mathbf{G}^{(1)}, \ldots, \mathbf{G}^{(L)}\}$, where

$$\mathbf{A}^{(l+1)} = P_E\left(\mathbf{A}^{(l)}\right) \tag{8}$$

$$\mathbf{V}^{(l+1)} = P_V\left(\mathbf{V}^{(l)}\right) \tag{9}$$

P_E is the edge pooling and P_V is the vertex pooling. The initial graph $\mathbf{G}^{(0)}$ is constructed by the HSI cube in set $\mathscr{X} = \{\mathbf{X}_1, \mathbf{X}_2, \ldots, \mathbf{X}_N\}$ based on Gaussian similarity, which has been discussed in the Section III-A. To build a graph pooling technique adapted to HSI, we clarify the spatial relationship of local regions. All the pixels in the local region serve to classify the center pixel, and generally, pixels further from the center contribute less. Harnessing such spatial relationships is one of the essential functions of our pooling operation. Some graph pooling strategies, like the one in [33], are not our option because they form completely new vertices, which lose the bijection between pixels and vertices. As new vertices lose their explicit meaning, it is difficult for them to express the local spatial relationship clearly. Therefore, we utilize graph pooling strategies that preserve the explicit meaning. Specifically, every vertex is matched with every pixel. We define Ω as the index set that contains all the subset pixels to be preserved around the center pixel

$$\mathbf{A}^{(l+1)} = \mathbf{A}^{(l)}_{\Omega,\Omega} \tag{10}$$

$$\mathbf{V}^{(l+1)} = \mathbf{V}^{(l)}_{\Omega}. \tag{11}$$

It is clear that the above graph pooling operation does not affect the correspondence between vertices and pixels. Moreover, we can easily design Ω with the guidance of the spatial relationship. A general intuition is that pixels that are far from the center tend to be removed. In this article, we directly preserve pixels in a subwindow. A square window of pixels centered at the center pixel is selected to form the next layer's feature matrix and adjacency matrix. An illustration of this spatial pooling method is shown in Fig. 3.

As mentioned in (6), $\mathbf{H}^{(l')} \in \mathbb{R}^{s\times s\times c_1}$ is the output of the lth graph convolution layer, where $s \times s$ is the spatial size and c_1 is the channel number. We reshape $\mathbf{H}^{(l')}$ to $\mathbf{H}^{(l')} \in \mathbb{R}^{n\times c_1}$, where $n = s \times s$. The spatial pooling method reduces the spatial size from $s \times s$ to $s_1 \times s_1$. While Ω consists of the subwindow pixels' indices, the adjacent matrix $\mathbf{A}^{(l)} \in \mathbb{R}^{n\times n}$ and the feature matrix $\mathbf{H}^{(l')} \in \mathbb{R}^{n\times c_1}$ are reduced correspondingly. Therefore, a subset of pixels $\mathbf{H}^{(l+1)} \in \mathbb{R}^{s_1\times s_1\times c_1}$ centered at the center pixel is selected

$$\mathbf{H}^{(l+1)} = \mathbf{H}^{(l')}_{\Omega}. \tag{12}$$

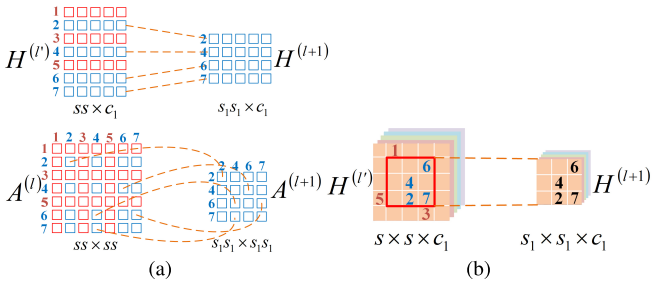


Fig. 3. Illustration of the proposed spatial pooling method. Vertices and edges in red are preserved in (a), which corresponds to pixels in red window in (b). (a) Pooling in graph domain. (b) Pooling in spatial domain.

Windows of different sizes will affect the performance, and a detailed discussion is shown in the experimental section.

C. SPGCN

The proposed spatial pooling graph convolution (SPGC) module is a combination of the spatial pooling method and the graph convolution method. Graph convolution layer fuse features between different pixels by the adjacency matrix. After that, the spatial pooling layer reduces the size of the HSI patch based on the spatial window.

In this section, we give a detailed analysis of the proposed SPGC module. We explain the function of the SPGC and demonstrate that SPGC performed a feature fusion operation between HSI patches of different spatial sizes.

We first substitute (7) into (6), and the graph convolution layer can be expressed as

$$\mathbf{H}^{(l')} = \sigma\left(\mathbf{D}^{(l)^{-1}}\mathbf{A}^{(l)}\mathbf{H}^{(l)}\mathbf{W}^{(l)} + \mathbf{H}^{(l)}\mathbf{W}^{(l)}\right) \tag{13}$$

$\mathbf{D}^{(l)^{-1}}\mathbf{A}^{(l)}$ is the row normalization of $\mathbf{A}^{(l)}$. Therefore, the graph convolution layer can be regarded as a summation of features obtained by graph propagation and features obtained by a fully connected layer in the spectral domain. We further substitute (13) into (12)

$$\mathbf{H}^{(l+1)} = \mathbf{H}^{(l')}_{\Omega} = \sigma\left(\mathbf{D}^{(l)^{-1}}\mathbf{A}^{(l)}\mathbf{H}^{(l)}\mathbf{W}^{(l)} + \mathbf{H}^{(l)}\mathbf{W}^{(l)}\right)_{\Omega}. \tag{14}$$

Based on (12), Ω is the indices of the pixels in a subwindow. Since window size $s \times s$ and $s_1 \times s_1$ are fixed, Ω is predetermined and has no relevance to the network's operation. Therefore, pooling operation can be performed before the activation function. In other words, pixels are selected separately on the two terms of (14)

$$\mathbf{H}^{(l+1)} = \sigma\left(\left(\mathbf{D}^{(l)^{-1}}\mathbf{A}^{(l)}\mathbf{H}^{(l)}\mathbf{W}^{(l)}\right)_{\Omega} + \left(\mathbf{H}^{(l)}\mathbf{W}^{(l)}\right)_{\Omega}\right). \tag{15}$$

Based on the knowledge of matrix operation, (15) can be further written as

$$\mathbf{H}^{(l+1)} = \sigma\left(\left(\mathbf{D}^{(l)^{-1}}\mathbf{A}^{(l)}\right)_{\Omega}\mathbf{H}^{(l)}\mathbf{W}^{(l)} + \mathbf{H}^{(l)}_{\Omega}\mathbf{W}^{(l)}\right). \tag{16}$$

That is to say, the spatial pooling operation is equivalent to vertex-selection on the normalized adjacency matrix $\mathbf{D}^{(l)^{-1}}\mathbf{A}^{(l)}$ and feature matrix $\mathbf{H}^{(l)}$. The overall architecture based on (16) is shown in Fig. 4. As is mentioned in (12), $(\cdot)_{\Omega}$ is the row selection operation that selects pixels inside subwindow of size $n_1 = s_1 \times s_1$. Therefore, $(\mathbf{D}^{(l)^{-1}}\mathbf{A}^{(l)})_{\Omega}$ is of size $n_1 \times n$, which indicates the adjacency relationship of pixels

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 15:07:25 UTC from IEEE Xplore. Restrictions apply.

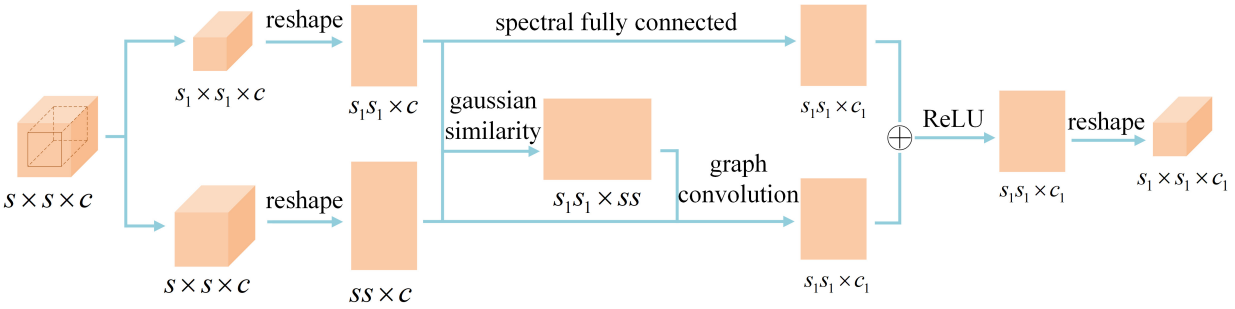


Fig. 4. Detailed analyzation of the SPGC module. The combination of graph convolution and spatial pooling result in a local feature fusion operation among different patches. $\oplus$ represents the matrix addition.

in subwindow $s_1 \times s_1$ and pixels in window $s \times s$. Then, as shown in Fig. 4, $(\mathbf{D}^{(l)^{-1}}\mathbf{A}^{(l)})_\Omega \mathbf{H}^{(l)}\mathbf{W}^{(l)}$ can be regarded as a feature fusion technique, which fuses features in window $s \times s$ to subwindow $s_1 \times s_1$. In addition, $\mathbf{H}_\Omega^{(l)}\mathbf{W}^{(l)}$ denotes the spectral fully connected layer on subwindow $s_1 \times s_1$.

In summary, $\mathbf{H}^{(l+1)} \in \mathbb{R}^{n_1 \times c_1}$ utilizes the information of $\mathbf{H}^{(l)} \in \mathbb{R}^{n \times c}$ in two ways. Graph convolution based on the nonsquare adjacency matrix $(\mathbf{D}^{(l)^{-1}}\mathbf{A}^{(l)})_\Omega \in \mathbb{R}^{n_1 \times n}$ fuses pixel features in window $s \times s$ to subwindow $s_1 \times s_1$. In addition, a spectral fully connected layer is applied on the subwindow $\mathbf{H}_\Omega^{(l)} \in \mathbb{R}^{n_1 \times c}$. Consequently, multilayer stacked SPGC modules gradually fuse features in large patches to small patches based on the fusion technique of graph convolution.

After multiple SPGC modules, a fully connected layer with a softmax classifier is applied to the output $\mathbf{H}^{(L)}$ to obtain the classification results. Finally, we use cross-entropy as the loss function for the classification of HSI

$$\mathbf{Z} = \text{softmax}\left(\mathbf{H}^{(L)}\mathbf{W} + b\right) \tag{17}$$

$$\mathbf{L} = -\sum_{y \in y_K}\sum_{f=1}^{C} \mathbf{Y}_f \log \mathbf{Z}_f \tag{18}$$

where the classification result $\mathbf{Z}$ is the output of the softmax classifier and C denotes the number of labels. y_K is the indices corresponding to the labeled pixels, and $\mathbf{Y}$ denotes the label matrix. Adam optimizer [48] is performed to learn the network parameters.

IV. Experiments

A. Data Description

The performance of the proposed SPGCN is evaluated on four benchmark HSI datasets, i.e., the Indian Pines, the University of Pavia, the Houston 2013, and the Salinas.

1) Indian Pines: The dataset was gathered by the Airborne Visible/Infrared Imaging Spectrometer (AVIRIS) over the Indian Pines test site in North-Western Indiana. It consists of 145 × 145 pixels with a spatial resolution of 20 m and 220 spectral bands in the wavelength ranging from 0.4 to 2.5 μm. As a usual step, 20 water absorption bands are removed and 200 bands are preserved. The dataset includes 16 classes with 10 249 labeled pixels. We randomly select 10%, 10%, and 80% of the whole labeled samples as the training, validation, and testing sets. Fig. 5 shows the false-color composite and the ground-truth map of the Indian Pines dataset. Table I lists the number of training, validation, and testing samples.

TABLE I
Number of Training, Validation, and Testing Samples on the Indian Pines

ID	Class Name	Train	Val	Test
1	Alfalfa	5	5	36
2	Corn-notill	143	143	1142
3	Corn-mintill	83	83	664
4	Corn	24	24	109
5	Grass-pasture	49	49	395
6	Grass-trees	73	73	584
7	Grass-pasture-mowed	3	3	22
8	Hay-windrowed	48	48	382
9	Oats	2	2	16
10	Soybean-notill	98	98	776
11	Soybean-mintill	246	246	1963
12	Soybean-clean	60	60	473
13	Wheat	21	21	163
14	Woods	127	127	1011
15	Buildings-Grass-Trees-Drives	39	39	308
16	Stone-Steel-Towers	10	10	73
	TOTAL	1031	1031	8187



Fig. 5. Indian Pines dataset. (a) False-color image. (b) Ground-truth. (c) Color bar.

2) University of Pavia: The dataset was acquired by the Reflective Optics System Imaging Spectrometer (ROSIS) sensor during a flight campaign over Pavia, north Italy. It consists of 610 × 340 pixels with a spatial resolution of 1.3 m by pixel. The number of spectral bands is 115 and the spectral resolution is 4 nm ranging from 0.43 to 0.86 μm: 12 noisy bands are removed and the remaining 103 bands are used for classification. The dataset includes nine classes with 42 776 labeled pixels. We randomly select 2%, 2%, and 96% of the whole labeled samples as the training, validation, and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 15:07:25 UTC from IEEE Xplore. Restrictions apply.

TABLE II

NUMBER OF TRAINING, VALIDATION, AND TESTING SAMPLES ON THE PAVIA UNIVERSITY

ID	Class Name	Train	Val	Test
1	Asphalt	133	133	6498
2	Meadows	373	373	18276
3	Gravel	42	42	2057
4	Trees	62	62	3002
5	Painted metal sheets	27	27	1318
6	Bare Soil	101	101	4928
7	Bitumen	27	27	1303
8	Self-Blocking Bricks	74	74	3608
9	Shadows	19	19	928
	TOTAL	858	858	41918

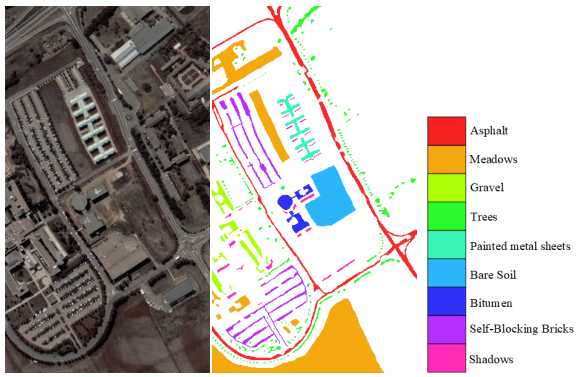


Fig. 6. University of Pavia dataset. (a) False-color image. (b) Ground-truth map. (c) Color bar.

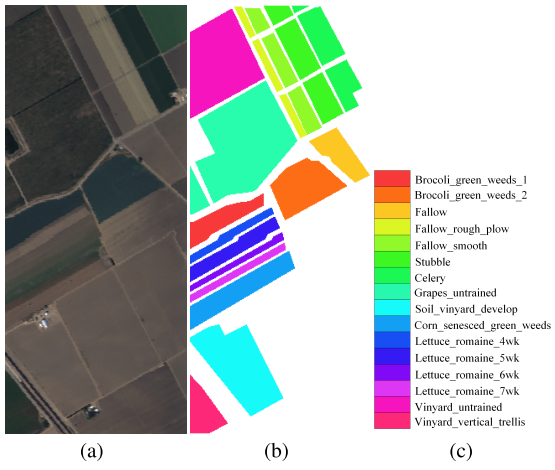


Fig. 7. Salinas dataset. (a) False-color image. (b) Ground-truth map. (c) Color bar.

testing sets. Fig. 6 shows the false-color composite and the ground-truth map of the University of Pavia dataset. Table II lists the number of training, validation, and testing samples.

3) Salinas: The dataset was gathered by the Airborne Visible/Infrared Imaging Spectrometer (AVIRIS) over Salinas Valley, California. It consists of 512 × 217 samples and the spatial resolution is 3.7 m by pixel. The number of spectral bands is 224 with wavelengths ranging from 0.4 to 2.5 μm. The dataset contains 16 classes with 54 129 labeled samples. Fig. 7 shows the false-color composite and the ground-truth map of the Salinas dataset. We randomly select 2%, 2%, and 96% of the whole labeled samples as the training, validation, and testing sets. Table III lists the number of training, validation, and testing samples.

TABLE III

NUMBER OF TRAINING, VALIDATION, AND TESTING SAMPLES ON THE SALINAS

ID	Class Name	Train	Val	Test
1	Brocoli_g_w_1	41	41	1968
2	Brocoli_g_w_2	75	75	3651
3	Fallow	40	40	1936
4	Fallow_r_p	28	28	1366
5	Fallow_s	54	54	2624
6	Stubble	80	80	3879
7	Celery	72	72	3507
8	Grapes_u	226	226	11045
9	Soil_v_d	125	125	6078
10	Corn_s_g_w	66	66	3212
11	Lettuce_r_4wk	22	22	1046
12	Lettuce_r_5wk	39	39	1888
13	Lettuce_r_6wk	19	19	897
14	Lettuce_r_7wk	22	22	1048
15	Vinyard_u	146	146	7122
16	Vinyard_v_t	37	37	1770
	TOTAL	1092	1092	53037

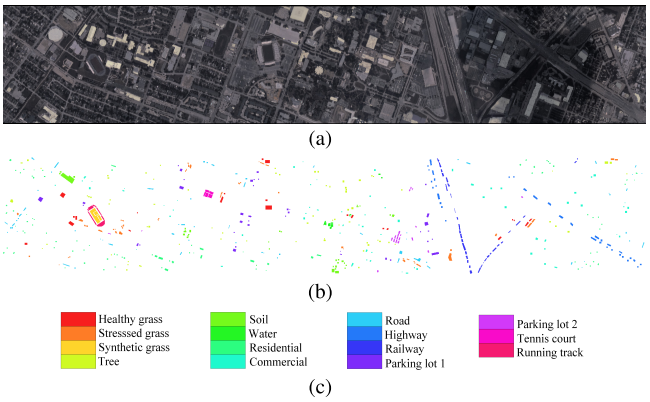


Fig. 8. Houston 2013 dataset. (a) False-color image. (b) Ground-truth map. (c) Color bar.

4) Houston 2013: The dataset was gathered over the University of Houston campus by the Compact Airborne Spectrographic Imager (CASI) sensor in June 2012. It consists of 349 × 1905 samples and the spatial resolution is 2.5 m by pixel. The number of spectral bands is 144 with wavelengths ranging from 0.38 to 1.05 μm. The dataset contains 15 classes with 15 029 labeled samples. Fig. 8 shows the false-color composite and the ground-truth map of the Salinas dataset. We randomly select 50 samples per class as the training and validation sets, while the rest are the test sets. Table IV lists the number of training, validation, and testing samples.

B. Basic Experimental Settings

To give a detailed evaluation of the proposed SPGCN model, experiments are implemented on the system of Windows 10 using Tensorflow with Adam optimizer. The hardware environment is a computer with an AMD Ryzen 5 2600 at 3.4 GHz × 6 with 16-GB RAM and an NVIDIA GeForce 1070 graphics processing unit (GPU) with 8-GB

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 15:07:25 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
NUMBER OF TRAINING, VALIDATION, AND TESTING SAMPLES ON THE HOUSTON 2013

ID	Class Name	Train	Val	Test
1	Healthy grass	50	50	1151
2	Stressed grass	50	50	1154
3	Synthetic grass	50	50	597
4	Tree	50	50	1144
5	Soil	50	50	1142
6	Water	50	50	225
7	Residential	50	50	1168
8	Commercial	50	50	1144
9	Road	50	50	1152
10	Highway	50	50	1227
11	Railway	50	50	1135
12	Parking lot 1	50	50	1133
13	Parking lot 2	50	50	369
14	Tennis court	50	50	328
15	Running track	50	50	560
	TOTAL	750	750	13529

RAM. To quantify the classification performance of SPGCN, we adopt the overall accuracy (OA), average accuracy (AA), kappa coefficient (kappa), and accuracy of each class.

Two architectures with different spatial sizes are evaluated. First, 9×9 is generally the proper input spatial size for CNN-based methods, so 9×9 is selected as the input size of our model, which makes our model acquire the same information as others, resulting in SPGCN-9. Second, some other GCN-based methods are not based on local regions, which do not apply to the above general rule. Also, the spatial pooling method makes our model more suitable under large input spatial size. To make a fair comparison, we also set the spatial input size to 21×21 to guarantee the performance of our model, which results in SPGCN-21. The dropout rate for each SPGC module is set to 0.5, and the hidden layer size is set to 64. To effectively extract spectral–spatial features, we gradually reduce the spatial size based on the spatial pooling layer. Specifically, the number of SPGC modules is set to 3, and the spatial reduction of each spatial pooling layer is set to 2 for SPGCN-9 and 6 for SPGCN-21. Therefore, the set of spatial size is $\{9 \times 9, 7 \times 7, 5 \times 5, 3 \times 3\}$ in SPGCN-9 and $\{21 \times 21, 15 \times 15, 9 \times 9, 3 \times 3\}$ in SPGCN-21, respectively. For SPGCN-21 on Houston 2013 dataset, only 2 SPGC modules are applied, which result in $\{21 \times 21, 9 \times 9, 3 \times 3\}$. A detailed structure of the proposed model is shown in Table V. Some adjustments are further adopted on each dataset to boost performance, which are summarized in Table VI. The average pooling is performed on all the datasets except the Pavia University. For the Pavia University and Salinas, edges are constructed between all the vertices. For the Indian Pines and Houston 2013, half of the edges with smaller weights are removed to reduce noise. The learning rate is set to 0.001, and early stopping is performed to adjust the training epoch dynamically. In most cases, the training will stop before 2000 epochs.

C. Comparison With Other Methods

In this section, we evaluate the proposed method with one spectral based method: 1-D CNN, one semi-supervised GCN-based method: MDGCN [36], and five spectral–spatial-based methods: 2-D CNN [18], 2-D CNN-GCN [40], 3-D CNN, DHCNet [49], and SSRN [20]. The 1-D CNN, 2-D CNN, and 2-D CNN-GCN adopt the same number of hidden layers, which is {32, 64, 128}. For the 2-D CNN, the architecture is the same as the one in [18] except that the spatial receptive field is set to 3×3 for each convolution layer. For the 2-D CNN-GCN, we adopt the FuNet-C in [40]. The 3-D CNN is gathered from SSRN. We remove the residual module in the SSRN and set the spatial receptive field to 3×3 for all the convolution layers in SSRN to obtain the 3-D CNN. For the MDGCN, the performance is largely affected by the superpixel segmentation, so the settings are as follows: For the Indian Pines dataset and Pavia University dataset, we use the segmentation provided by the author; For the Salinas dataset and Houston 2013 dataset, a relatively large number of superpixels are segmented to guarantee the performance (nearly 4000 superpixels for Salinas and around 1000 for Houston 2013). All the methods are implemented with Keras or TensorFlow. For all the spectral–spatial-based methods, the number of epochs is set to 200 and the input spatial size is set to 9×9. Weight parameters of all methods are optimized using the Adam optimizer.

The first experiment is conducted on the Indian Pines dataset. The quantitative results obtained by different methods are summarized in Table VII. It can be seen that all the spectral–spatial-based methods achieve better performance compared to 1-D CNN. Meanwhile, compared with other comparison methods, the proposed SPGCN-9 achieves the highest performance in terms of OA, AA, and Kappa, and SPGCN-21 further improves the OA and Kappa results, which

TABLE V
DETAILED STRUCTURES OF THE SPGCN FRAMEWORK

Layers		SPGCN-9 Output patch size	SPGCN-21 Output patch size
Input		$9 \times 9 \times m$	$21 \times 21 \times m$
SPGC module	GCN	$9 \times 9 \times 64$	$21 \times 21 \times 64$
	Spatial Pooling	$7 \times 7 \times 64$	$15 \times 15 \times 64$
	Dropout	0.5	0.5
SPGC module	GCN	$7 \times 7 \times 64$	$15 \times 15 \times 64$
	Spatial Pooling	$5 \times 5 \times 64$	$9 \times 9 \times 64$
	Dropout	0.5	0.5
SPGC module	GCN	$5 \times 5 \times 64$	$9 \times 9 \times 64$
	Spatial Pooling	$3 \times 3 \times 64$	$3 \times 3 \times 64$
	Dropout	0.5	0.5
Average Pooling layer		$1 \times 1 \times 64$	$1 \times 1 \times 64$
FC layer		$1 \times c$	$1 \times c$
Softmax		$1 \times c$	$1 \times c$

1. m represents the number of HSI bands.
2. c represents the number of labels.
3. FC layer represents the fully connected layer.

TABLE VI
ADJUSTMENTS OF SPGCN ON EACH DATATSET

	Indian	Pavia	Salinas	Houston
Average Pooling	√	×	√	√
Edge Removing	√	×	×	√

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 15:07:25 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
CLASSIFICATION RESULTS BY DIFFERENT METHODS ON THE INDIAN PINES DATASET

Classes	1-D CNN	2-D CNN	2-D CNN-GCN	3-D CNN	DHCNet	MDGCN	SSRN	SPGCN-9	SPGCN-21
1	13.67 ± 9.32	72.46 ± 19.14	74.58 ± 14.34	98.61 ± 4.60	87.80 ± 13.02	78.79 ± 14.39	**99.14 ± 2.17**	97.22 ± 2.77	97.65 ± 1.86
2	80.51 ± 2.48	95.42 ± 2.15	95.26 ± 1.38	94.93 ± 2.60	96.23 ± 0.83	94.61 ± 2.48	97.25 ± 3.62	97.90 ± 1.26	**98.33 ± 0.89**
3	70.20 ± 3.67	95.36 ± 2.43	92.86 ± 2.76	95.49 ± 4.81	96.31 ± 3.21	95.54 ± 2.56	95.42 ± 8.39	97.45 ± 1.72	**98.32 ± 1.86**
4	64.26 ± 6.85	86.78 ± 4.64	83.56 ± 5.78	96.06 ± 3.14	94.77 ± 2.29	92.97 ± 5.54	**98.40 ± 1.98**	95.71 ± 2.64	98.19 ± 1.94
5	86.89 ± 1.81	95.71 ± 2.21	94.75 ± 3.71	98.36 ± 1.66	97.46 ± 1.74	95.38 ± 1.86	98.08 ± 1.14	**98.18 ± 0.96**	97.60 ± 2.44
6	97.15 ± 1.32	99.49 ± 0.26	98.77 ± 0.77	98.68 ± 1.27	97.99 ± 1.12	98.74 ± 0.52	98.65 ± 0.85	99.46 ± 0.58	**99.47 ± 0.56**
7	65.21 ± 12.2	69.33 ± 11.23	80.33 ± 11.01	**100 ± 0.00**	86.00 ± 9.79	85.83 ± 3.37	**100 ± 0.00**	97.27 ± 4.63	96.64 ± 4.13
8	98.76 ± 1.56	**100 ± 0.00**	**100 ± 0.00**	95.67 ± 2.05	98.68 ± 1.81	99.74 ± 0.46	99.61 ± 0.69	**100 ± 0.00**	**100 ± 0.00**
9	35.41 ± 16.00	67.45 ± 26.96	76.38 ± 20.55	**100 ± 0.00**	88.88 ± 10.76	87.60 ± 16.62	53.33 ± 49.88	97.50 ± 3.06	87.17 ± 16.53
10	78.76 ± 10.07	91.16 ± 3.42	94.45 ± 1.88	96.48 ± 2.05	95.79 ± 1.93	93.41 ± 3.40	96.55 ± 3.48	**97.19 ± 1.64**	97.17 ± 1.80
11	80.88 ± 2.13	95.08 ± 1.85	95.32 ± 2.32	96.55 ± 2.28	97.55 ± 1.03	97.47 ± 1.21	**99.32 ± 0.49**	99.27 ± 0.29	99.01 ± 0.78
12	78.11 ± 3.62	86.54 ± 5.42	86.25 ± 7.07	93.85 ± 6.39	91.53 ± 4.42	91.50 ± 3.63	97.70 ± 1.10	**98.68 ± 0.53**	97.52 ± 1.34
13	98.46 ± 1.42	99.92 ± 0.19	99.63 ± 0.46	**100 ± 0.00**	98.84 ± 1.03	99.03 ± 1.22	99.93 ± 0.24	99.69 ± 0.41	99.90 ± 0.22
14	95.55 ± 1.67	98.51± 1.04	97.54 ± 1.47	98.86 ± 0.97	97.84 ± 1.65	99.32 ± 0.58	99.42 ± 0.59	99.51 ± 0.45	**99.97 ± 0.09**
15	60.33 ± 5.79	88.34 ± 5.95	91.85 ± 5.35	94.02 ± 3.85	93.29 ± 2.14	97.13 ± 2.36	98.81 ± 1.26	98.43 ± 1.83	**99.63 ± 0.68**
16	90.62 ± 1.73	95.52 ± 3.26	96.88 ± 4.21	98.13 ± 1.41	97.28 ± 1.78	93.04 ± 4.58	96.68 ± 3.38	99.31 ± 1.26	**99.56 ± 1.54**
OA(%)	82.98 ± 0.28	94.70 ± 0.58	94.74 ± 0.71	96.41 ± 0.73	96.59 ± 0.86	96.46 ± 0.43	98.06 ± 1.52	98.60 ± 0.28	**98.71 ± 0.35**
AA(%)	76.38 ± 3.19	89.73 ± 1.87	91.15 ± 1.90	97.23 ± 0.70	94.78 ± 1.38	94.02 ± 1.55	95.52 ± 3.31	**98.29 ± 0.49**	97.89 ± 0.87
$\mathscr{K} \times 100$	80.01 ± 0.35	93.86 ± 0.65	93.90 ± 0.82	95.90 ± 0.83	96.12 ± 0.98	95.97 ± 0.49	97.80 ± 1.71	98.41 ± 0.32	**98.53 ± 0.40**

TABLE VIII
CLASSIFICATION RESULTS BY DIFFERENT METHODS ON THE UNIVERSITY OF PAVIA DATASET

Classes	1-D CNN	2-D CNN	2-D CNN-GCN	3-D CNN	DHCNet	MDGCN	SSRN	SPGCN-9	SPGCN-21
1	91.86 ± 0.78	95.74 ± 0.93	95.69 ± 1.13	96.84 ± 1.29	97.18 ± 1.16	99.15 ± 0.64	98.66 ± 1.32	98.61 ± 0.55	**99.18 ± 0.51**
2	98.88 ± 0.24	99.87 ± 0.05	99.68 ± 0.09	99.56 ± 0.23	99.62 ± 0.20	**99.98 ± 0.02**	99.08 ± 2.05	99.88 ± 0.12	99.89 ± 0.08
3	63.58 ± 3.22	82.39 ± 2.69	82.12 ± 4.65	95.72 ± 2.89	91.76 ± 3.00	98.53 ± 1.53	**99.08 ± 9.07**	93.59 ± 1.84	95.87 ± 2.16
4	95.45 ± 0.40	97.38 ± 0.66	98.33 ± 0.45	**99.41 ± 0.72**	97.59 ± 0.51	96.63 ± 1.49	93.47 ± 11.80	96.66 ± 1.06	96.54 ± 1.00
5	99.66 ± 0.22	99.04 ± 1.07	99.82 ± 0.27	99.93 ± 0.08	99.68 ± 0.45	98.02 ± 1.17	99.85 ± 0.13	99.91 ± 0.10	**99.96 ± 0.10**
6	87.25 ± 1.96	95.56 ± 1.28	95.36 ± 1.84	99.13 ± 0.27	99.02 ± 1.40	99.65 ± 0.45	97.15 ± 6.08	99.63 ± 0.38	**99.68 ± 0.59**
7	93.65 ± 0.91	87.58 ± 2.07	83.81 ± 1.55	98.53 ± 1.67	89.25 ± 3.28	98.92 ± 1.69	**99.92 ± 0.16**	98.81 ± 1.41	99.15 ± 0.99
8	95.04 ± 1.27	92.56 ± 2.24	88.83 ± 4.17	92.39 ± 2.51	93.87 ± 2.07	97.50 ± 1.21	94.24 ± 7.95	96.38 ± 1.50	**99.07 ± 0.63**
9	99.90 ± 0.10	98.02 ± 0.51	**99.98 ± 0.04**	99.95 ± 0.08	99.74 ± 0.31	97.05 ± 0.91	99.65 ± 0.49	99.82 ± 0.14	99.93 ± 0.09
OA(%)	94.00 ± 0.29	96.61 ± 0.40	96.27 ± 0.36	98.21 ± 0.27	97.83 ± 0.32	99.13 ± 0.18	97.42 ± 2.05	98.79 ± 0.17	**99.22 ± 0.17**
AA(%)	91.70 ± 0.50	94.24 ± 0.63	93.86 ± 0.28	97.94 ± 0.42	96.41 ± 0.45	98.38 ± 0.29	97.51 ± 2.23	98.15 ± 0.26	**98.81 ± 0.29**
$\mathscr{K} \times 100$	91.40 ± 0.42	95.46 ± 0.54	95.02 ± 0.48	97.63 ± 0.36	97.12 ± 0.43	98.85 ± 0.24	96.60 ± 2.68	98.34 ± 0.30	**98.97± 0.22**

demonstrates the effectiveness of graph convolution in extracting local features. Due to the imbalance pixels of different land-cover categories, only 5, 3, and 2 labeled samples are selected as training samples in the "Alfalfa," "Grass-p-m," and "Oats" categories, which lead to relative instability in the classification results. For example, for the "Oats" category, all the methods except 3-D CNN and SPGCN-9 have variances of more than 10, and SSRN has a variance of nearly 50. As a result, SSRN performs poorly on the "Oat" categories, which is 53.33%. SPGCN-21 is also affected, which results in a lower AA compared to SPGCN-9. Nevertheless, SPGCN-21's AA is still higher than all other comparison methods due to the superior overall performance. In addition, by comparing the 2-D CNN-GCN with the 2-D CNN, the GCN module improves the "Alfalfa," of 2-D CNN by 2.9%, the "Grass-p-m" of 2-D CNN by 15.8%, the "Oats" by 13.2%. This indicates that by conducting graph propagation, GCN can effectively deal with the imbalance of training samples in different categories.

The second experiment is performed on the University of Pavia. Results obtained by different methods are summarized in Table VIII. MDGCN and SPGCN-21 outperform all other methods to a certain extent in terms of OA, AA, and Kappa, and SPGCN-21 achieves the highest. For all other spectral–spatial-based methods with 9×9 input spatial size, SPGCN-9 achieves the highest in terms of OA, AA, and Kappa. On the other hand, 3-D CNN surpasses SSRN, which is mainly due to the category "Trees," where 3-D CNN achieves an accuracy of 99.41%, compared with 93.47% for SSRN and 96.66% for SPGCN-9. Note that 3-D CNN is obtained by removing the residual module from SSRN. Category "Trees" are small dote-like objects in the University of Pavia dataset with distinctive features. Therefore, shallow neural networks with limited input spatial size are enough to distinguish

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 15:07:25 UTC from IEEE Xplore. Restrictions apply.

TABLE IX
CLASSIFICATION RESULTS BY DIFFERENT METHODS ON THE SALINAS DATASET

Classes	1-D CNN	2-D CNN	2-D CNN-GCN	3-D CNN	DHCNet	MDGCN	SSRN	SPGCN-9	SPGCN-21
1	98.70 ± 0.55	99.88 ± 0.19	97.49 ± 2.46	98.92 ± 3.02	98.47 ± 1.47	**100 ± 0.00**	**100 ± 0.00**	**100 ± 0.00**	**100 ± 0.00**
2	99.79 ± 0.15	99.97 ± 0.05	**100 ± 0.00**	97.08 ± 8.27	99.64 ± 0.40	99.96 ± 0.09	99.95 ± 0.09	99.97 ± 0.04	**100 ± 0.00**
3	92.94 ± 1.50	99.17 ± 0.20	98.63 ± 1.32	99.13 ± 0.95	99.08 ± 0.54	99.77 ± 0.69	**100 ± 0.00**	99.89 ± 0.19	99.99 ± 0.01
4	98.89 ± 0.70	99.28 ± 0.75	**99.80 ± 0.19**	99.08 ± 0.87	98.55 ± 0.75	98.79 ± 0.77	99.55 ± 0.29	99.34 ± 0.32	99.00 ± 0.82
5	98.34 ± 0.56	97.81 ± 0.33	97.76 ± 0.65	99.10 ± 0.54	98.78 ± 0.88	97.56 ± 0.32	**99.87 ± 0.05**	98.88 ± 0.97	98.97 ± 0.62
6	99.84 ± 0.10	**100 ± 0.00**	99.98 ± 0.02	99.91 ± 0.11	99.94 ± 0.07	99.69 ± 0.11	**100 ± 0.00**	99.97 ± 0.03	99.97 ± 0.02
7	99.43 ± 0.10	99.73 ± 0.32	99.30 ± 0.22	99.93 ± 0.09	99.55 ± 0.29	99.51 ± 0.17	**100 ± 0.00**	99.92 ± 0.06	99.96 ± 0.02
8	79.58 ± 5.25	87.61 ± 3.27	87.90 ± 3.25	92.17 ± 4.25	92.47 ± 6.42	**98.99 ± 0.89**	95.26 ± 6.71	96.66 ± 2.09	98.91 ± 0.54
9	99.82 ± 0.14	99.94 ± 0.11	99.54 ± 0.38	99.61 ± 0.18	99.18 ± 1.22	99.88 ± 0.11	99.97 ± 0.05	**100 ± 0.00**	**100 ± 0.00**
10	90.78 ± 1.24	95.75 ± 1.09	95.74 ± 1.58	98.74 ± 1.47	97.08 ± 1.60	97.67 ± 1.07	**99.81 ± 0.19**	97.99 ± 1.13	98.86 ± 0.75
11	90.79 ± 1.46	98.62 ± 0.98	96.32 ± 4.18	97.59 ± 1.91	94.36 ± 4.11	97.52 ± 1.80	**99.75 ± 0.43**	99.41 ± 0.45	99.27 ± 0.75
12	99.39 ±0.39	99.86 ± 0.29	99.87 ± 0.25	99.51 ± 0.41	99.76 ± 0.50	**100 ± 0.00**	**100 ± 0.00**	99.89 ± 0.11	99.80 ± 0.29
13	98.48 ± 0.63	99.32 ± 0.43	98.25 ± 0.83	98.90 ± 0.91	99.46 ± 0.31	98.27 ± 0.91	99.93 ± 0.05	**99.94 ± 0.13**	99.93 ± 0.19
14	95.09 ± 1.33	98.31 ± 1.47	96.84 ± 0.89	99.54 ± 0.61	98.31 ± 0.59	96.26 ± 1.79	**99.83 ± 0.33**	99.38 ± 0.47	99.43 ± 0.44
15	75.22 ± 11.31	87.40 ± 5.41	89.31 ± 3.53	87.81 ± 8.70	87.79 ± 10.32	98.82 ± 0.34	96.37 ± 3.53	95.89 ± 1.53	**98.98 ± 0.70**
16	97.90 ± 0.64	97.81 ± 1.33	98.58 ± 0.47	99.15 ± 0.80	96.14 ± 2.40	98.09 ± 0.95	**100 ± 0.00**	98.70 ± 0.69	99.15 ± 0.30
OA(%)	90.49 ± 0.23	95.13 ± 0.27	95.21 ± 0.38	95.71 ± 1.62	95.98 ± 0.87	99.04 ± 0.18	98.27 ± 1.42	98.47 ± 0.29	**99.42 ± 0.15**
AA(%)	94.47 ± 0.19	97.53 ± 0.20	97.21 ± 0.27	97.88 ± 0.65	97.41 ± 0.47	98.80 ± 0.16	99.39 ± 0.34	99.11 ± 0.10	**99.52 ± 0.10**
$\mathscr{K} \times 100$	89.39 ± 0.25	94.57 ± 0.30	94.66 ± 0.42	95.23 ± 1.79	95.53 ± 0.97	98.93 ± 0.20	98.07 ± 1.58	98.30 ± 0.32	**99.36 ± 0.17**

TABLE X
CLASSIFICATION RESULTS BY DIFFERENT METHODS ON THE HOUSTON 2013 DATASET

Classes	1-D CNN	2-D CNN	2-D CNN-GCN	3-D CNN	DHCNet	MDGCN	SSRN	SPGCN-9	SPGCN-21
1	**98.41 ± 0.84**	95.63 ± 2.54	94.88 ± 2.94	96.17 ± 2.71	95.48 ± 2.75	95.91 ± 2.65	97.97 ± 2.55	98.45 ± 0.97	97.44 ± 1.74
2	92.71 ± 4.01	97.66 ± 1.84	**99.25± 0.83**	94.63 ± 6.52	98.60 ± 1.57	94.38 ± 2.04	98.42 ± 1.96	98.91 ± 1.00	99.23 ± 0.88
3	96.68 ± 4.45	99.78 ± 0.13	99.70 ± 0.78	99.79 ± 0.53	99.76 ± 0.46	**100 ± 0.00**	99.95 ± 0.17	**100 ± 0.00**	99.96 ± 0.06
4	91.30 ± 1.81	95.97 ± 2.17	97.48 ± 1.55	98.39 ± 2.51	95.63 ± 2.07	98.75 ± 1.69	98.33 ± 3.18	98.12 ± 2.18	**99.31 ± 0.95**
5	98.23 ± 1.05	99.33 ± 0.50	99.34 ± 0.64	97.95 ± 2.20	98.52 ± 1.04	99.88 ± 0.39	98.84 ± 1.59	**99.97 ± 0.04**	99.91 ± 0.02
6	97.76 ± 2.04	95.67 ± 2.71	94.04 ± 3.90	97.08 ± 3.58	97.25 ± 2.59	97.77 ± 1.29	98.79 ± 1.95	**99.02 ± 1.24**	98.70 ± 1.04
7	76.85 ± 4.08	90.31 ± 2.83	87.79 ± 2.14	96.55 ± 1.77	91.00 ± 3.07	94.60 ± 1.52	**96.75 ± 2.12**	93.86 ± 3.48	96.63± 1.59
8	59.22 ± 7.86	81.82 ± 3.67	83.75 ± 4.42	96.22 ± 3.04	87.03 ± 5.42	87.78 ± 4.37	**98.69 ± 2.10**	89.80 ± 2.95	90.51 ± 2.50
9	70.11 ± 12.07	86.72 ± 3.47	84.11 ± 2.81	95.53 ± 2.10	88.07 ± 2.96	91.62 ± 4.34	**95.60 ± 3.37**	91.11 ± 3.44	94.82 ± 2.33
10	81.37 ± 7.78	97.11 ± 1.92	94.34 ± 2.17	93.81 ± 2.53	94.11 ± 5.18	98.19 ± 1.84	96.21 ± 2.63	98.56 ± 2.16	**99.74 ± 0.24**
11	73.23 ± 6.41	93.71 ± 2.32	90.37 ± 2.95	95.50 ± 4.59	92.09 ± 2.45	97.48 ± 2.98	96.41 ± 1.93	**98.07 ± 1.78**	97.81 ± 1.97
12	67.06 ±11.53	95.64 ± 1.59	95.41 ± 1.90	92.30 ± 3.43	96.05 ± 1.75	92.80 ± 4.63	97.42 ± 0.99	**97.91 ± 1.32**	97.77 ± 1.59
13	52.70 ± 3.90	95.90 ± 0.81	94.83 ± 1.72	92.77 ± 4.62	**97.16 ± 1.98**	94.35 ± 3.52	91.89 ± 5.76	92.39 ± 2.22	95.46 ± 2.07
14	99.23 ± 0.57	99.90 ± 0.26	99.80 ± 0.41	99.92 ± 0.16	99.87 ± 0.39	**100 ± 0.00**	99.02 ± 1.49	**100 ± 0.00**	**100 ± 0.00**
15	98.96 ± 0.39	99.87 ± 0.32	99.97 ± 0.05	97.69 ± 1.74	99.22 ± 1.31	**100 ± 0.00**	98.59 ± 0.90	**100 ± 0.00**	99.89 ± 0.14
OA(%)	82.76 ± 1.00	94.23 ± 0.53	93.56 ± 0.33	95.85 ± 1.03	94.48 ± 0.78	95.71 ± 0.65	97.44 ± 0.63	96.77 ± 0.33	**97.58 ± 0.24**
AA(%)	84.00 ± 0.85	95.00 ± 0.50	94.25 ± 0.44	96.28 ± 0.86	95.24 ± 0.63	96.23 ± 0.51	97.53 ± 0.47	97.03 ± 0.24	**97.82 ± 0.17**
$\mathscr{K} \times 100$	81.26 ± 1.09	93.92 ± 2.54	93.13 ± 0.52	95.51 ± 1.11	94.03 ± 0.84	95.36 ± 0.70	97.23 ± 0.68	96.50 ± 0.36	**97.38 ± 0.26**

them correctly. In addition, no significant difference is found when comparing 2-D CNN-GCN and 2-D CNN. The possible explanation for this phenomenon is that the Pavia University dataset has a relatively balanced training category distribution compared with the Indian Pines dataset.

The third experiment is conducted on the Salinas. The quantitative results obtained by different methods are summarized in Table IX. Again, SPGCN-21 and MDGCN surpass all other results in terms of OA and Kappa, while SPGCN-21 is much higher. SPGCN-21 surpasses all other methods by a large margin. Note that the AA of SSRN is very high, which surpasses all other methods except SPGCN-21. SSRN also achieves the best results in over half of the classes. Despite this, SPGCN-9 still surpasses SSRN in terms of OA and Kappa, which is mainly due to the category "Grapes_u": SPGCN-9 achieves an accuracy of 96.66%, compared with 95.26% for SSRN. This is the largest flat area in the Salinas. This result shows that GCN can extract large spectral–spatial features as effectively as CNN-based methods.

The fourth experiment is conducted on the Houston 2013 dataset and results are summarized in Table X. Note that the set of spatial size in SPGCN-21 for Houston 2013 is

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 15:07:25 UTC from IEEE Xplore. Restrictions apply.

TABLE XI
TRAINING TIME, TESTING TIME (SECONDS), AND NUMBER OF PARAMETERS OF ALL METHODS ON EACH DATASET

	Methods	1-D CNN	2-D CNN	2-D CNN-GCN	3-D CNN	DHCNet	MDGCN	SSRN	SPGCN-9	SPGCN-21
Indian	Training time	22.79	25.10	42.18	228.77	55.83	18.20	429.13	149.05	1117.67
	Testing time	0.86	0.73	3.33	2.40	2.88	2.19	4.55	3.79	44.34
	Parameters(K)	38.30	152.50	212.03	2712.49	1308.18	13.07	141.83	22.61	22.61
Pavia	Training time	21.43	18.90	31.66	116.55	52.31	79.81	210.93	141.16	672.81
	Testing time	7.69	1.46	5.57	6.86	8.44	7.34	12.64	7.55	143.24
	Parameters(K)	30.29	123.66	170.58	1385.21	1229.70	6.81	92.50	20.55	20.55
Salinas	Training time	25.4	26.13	46.40	245.32	56.42	808.15	470.53	297.56	2171.68
	Testing time	5.56	6.54	10.89	20.9	12.06	30.76	35.81	10.91	216.07
	Parameters(K)	38.56	153.65	213.70	2767.79	1311.64	13.31	143.88	22.86	22.86
Houston	Training time	16.42	20.37	20.10	120.76	52.75	20.89	167.68	446.04	1734.63
	Testing time	1.18	3.21	2.14	2.78	3.16	1.45	3.83	5.69	58.58
	Parameters(K)	34.46	136.24	188.50	2712.49	1259.60	9.65	113.14	18.96	14.67

TABLE XII
DIFFERENT SPATIAL SIZES OF SPGCN

Layers	$d = 2$	$d = 4$	$d = 6$	$d = 8$
SPGCN	9×9	15×15	21×21	27×27
SPGCN	7×7	11×11	15×15	19×19
SPGCN	5×5	7×7	9×9	11×11
FC	3×3	3×3	3×3	3×3

TABLE XIII
DIFFERENT LAYER NUMBERS OF SPGCN

Layers	$l = 2$	$l = 3$	$l = 4$
SPGCN	-	-	21×21
SPGCN	-	21×21	17×17
SPGCN	21×21	15×15	13×13
SPGCN	9×9	9×9	9×9
FC	3×3	3×3	3×3

$\{21 \times 21, 9 \times 9, 3 \times 3\}$. It can be seen that SPGCN-21 achieves the highest performance in terms of OA, AA, and Kappa, while SSRN surpasses SPGCN-9 and is better than any other methods except SPGCN-21. The Houston 2013 is a relatively large dataset with complex background and small scattered labeled areas. Therefore, the SPGCN-21 first fuses features of nearby backgrounds to the small area with the first SPGC module, and the second SPGC module combines background features with target features to perform classification. The feature extracted by SPGCN-9 lacks robust background information due to the limited input spatial size. It is noticed that, although SSRN cannot handle large spatial sizes, it can extract comprehensive local features based on the 3-D CNN and residual blocks.

Figs. 9–12 visualize the false-color images, the corresponding ground truth maps, and the classification results obtained by different methods on the three datasets. 1-D CNN has pepper-noise-like mistakes on the three datasets, while all the spectral–spatial based methods generate more smooth classification maps. On the Indian Pines dataset, the proposed method does not have any large misclassification area compared with other methods, and only some dot-like mistakes occurred on the boundary. On the University of Pavia and Salinas dataset, the proposed SPGCN shows similar results compared with MDGCN and SSRN, demonstrating that GCN can extract spectral–spatial features as efficiently as CNN.

The training time, the testing time, and the number of parameters are listed in Table XI. To speed up our model, all the adjacent matrices for the training set are precomputed before training to avoid repeat calculations. For the proposed methods, SPGCN has the least amount of parameters except MDGCN, which is even less than 1-D CNN. Also, SPGCN-21 and SPGCN-9 have the same number of parameters on all the datasets except the Houston 2013, because the hidden dimensions and hidden layers are the same. For Houston 2013, SPGCN-21 only has 2 SPGC modules, resulting in a smaller number of parameters than SPGCN-9. On the other hand, the training and testing time for SPGCN-21 and SPGCN-9 is relatively long, while SPGCN-21 achieves the longest. This is due to the high computation cost of every local adjacent matrix. Still, the training time of SPGCN-9 is shorter on three of the four datasets compared with SSRN, which demonstrates that SPGCN is comparable to SSRN when the input spatial sizes are the same. For other methods, MDGCN has the smallest number of parameters due to the simple and shallow structure. The training and testing are also fast except on the Salinas dataset (808.15 s), because we use nearly 4000 superpixels to exaggerate the performance. 3-D CNN and DHCNet have much more parameters than other methods, and 3-D CNN has the most. 3-D CNN is obtained by removing the residual module in SSRN and set all the spatial receptive fields to 3×3. Therefore, although 3-D CNN is shallow than SSRN due to the removing operation, the change of spatial receptive field from 1×1 to 3×3 greatly increases the number of parameters.

D. Impact of Spatial-Window Size

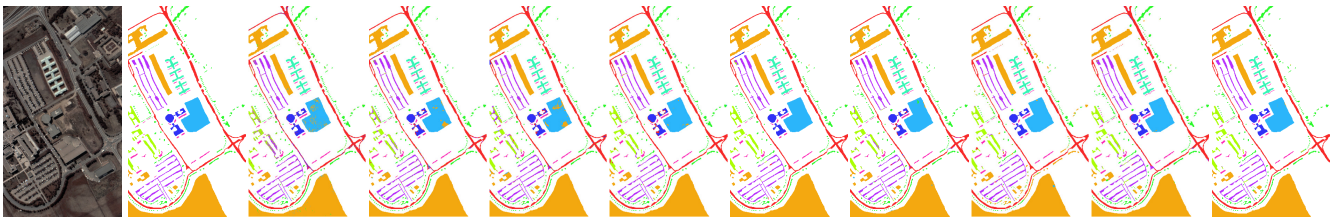
It is known that HSIs are locally similar. Therefore, a proper size of the spatial window can improve the classification performance significantly. Since the spatial sizes of all SPGC modules are adjustable, a large number of spatial-size combinations can be generated. However, too much spatial reduction in one pooling layer can lead to excessive information loss.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 15:07:25 UTC from IEEE Xplore. Restrictions apply.



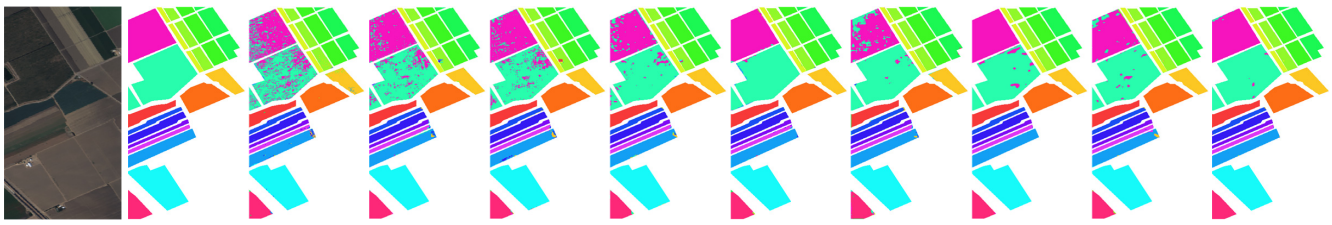
(a) (b) (c) (d) (e) (f) (g) (h) (i) (j) (k)

Fig. 9. Classification results obtained by all methods for Indian Pines. (a) False color image. (b) Ground truth. (c) 1-D CNN. (d) 2-D CNN. (e) 2-D CNN-GCN. (f) 3-D CNN. (g) MDGCN. (h) DHCNet. (i) SSRN. (j) SPGCN-9. (k) SPGCN-21.



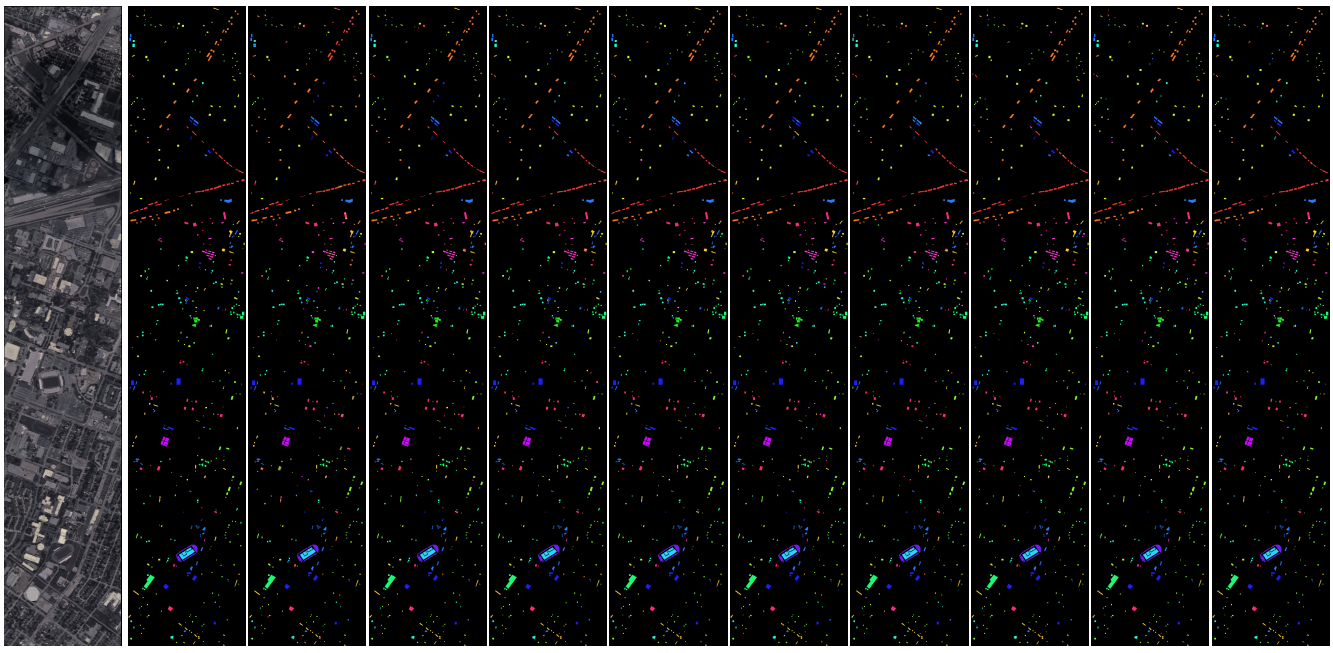
(a) (b) (c) (d) (e) (f) (g) (h) (i) (j) (k)

Fig. 10. Classification results obtained by all methods for University of Pavia. (a) False color image. (b) Ground truth. (c) 1-D CNN. (d) 2-D CNN. (e) 2-D CNN-GCN. (f) 3-D CNN. (g) MDGCN. (h) DHCNet. (i) SSRN. (j) SPGCN-9. (k) SPGCN-21.



(a) (b) (c) (d) (e) (f) (g) (h) (i) (j) (k)

Fig. 11. Classification results obtained by all methods for Salinas. (a) False color image. (b) Ground truth. (c) 1-D CNN. (d) 2-D CNN. (e) 2-D CNN-GCN. (f) 3-D CNN. (g) MDGCN. (h) DHCNet. (i) SSRN. (j) SPGCN-9. (k) SPGCN-21.



(a) (b) (c) (d) (e) (f) (g) (h) (i) (j) (k)

Fig. 12. Classification results obtained by all methods for Houston 2013. (a) False color image. (b) Ground truth. (c) 1-D CNN. (d) 2-D CNN. (e) 2-D CNN-GCN. (f) 3-D CNN. (g) MDGCN. (h) DHCNet. (i) SSRN. (j) SPGCN-9. (k) SPGCN-21.

In this article, we gradually reduce the spatial size layer by layer in the forward propagation. We set d as the reduction of spatial size in one SPGC module. d is set to 2, 4, 6, and 8, and we fix the output spatial size of the last SPGC module to be 3×3. This results in different spatial sizes of each layer, and a detailed display is summarized in Table XII.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 15:07:25 UTC from IEEE Xplore. Restrictions apply.

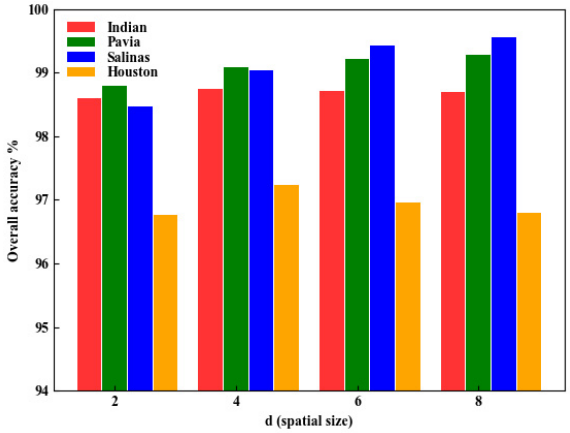


Fig. 13. OA (%) of SPGCN with different spatial sizes on the Indian, Pavia, Salinas, and Houston datasets.

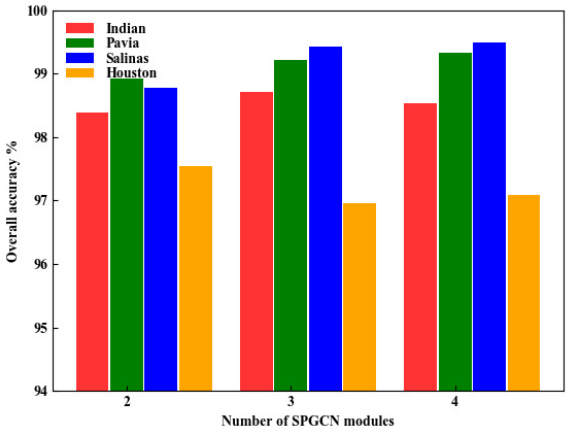


Fig. 14. OA (%) of SPGCN with different layer numbers on the Indian, Pavia, Salinas, and Houston datasets.

The four datasets in the previous section with the same settings are used to conduct experiments, which are shown in Figs. 5–8 and Tables I–IV. The OAs of different d are shown in Fig. 13. The smallest input patch size is 9×9, which corresponds to $d = 2$, while the largest input patch size is 27×27. In general, proper spatial size is essential to achieve a high OA, as it struck a balance between redundancy and insufficiency of information. It can be seen that on two of the four datasets, OAs continue to increase with the increase of d. On the Indian Pines dataset, OA remains stable after d reaches 4. While on the Houston 2013 dataset, OA achieves the highest when d is 4. Although 27×27 is a relatively large spatial size, the margin between the highest and lowest is less than one percent. This suggests that our model has the ability to harness large spatial input size, which can be explained by the combination of graph propagation and spatial pooling. All pixels establish similarities with the center pixel based on the Gaussian similarity. As SPGC module naturally fuses features from large patches to small patches, less related pixels will contribute little to the classification results. Therefore, redundant information is largely reduced.

E. Impact of Layer Numbers

As is mentioned in Section I, our SPGC module can be stacked to several layers. Therefore, the number of layers is a critical part for boosting classification performance. We conduct the experiments on the four datasets with three different numbers of SPGC module, i.e., $l = 2$, $l = 3$, and $l = 4$. The detailed spatial sizes of different layers are shown in Table XIII and experimental results are shown in Fig. 14. It can be observed that the appropriate number of layers varies with the dataset. OAs increase on the Pavia University and Salinas datasets when l ranges from 2 to 4. However, OAs on the Indian Pines decreased when l ranges from 3 to 4, whereas OAs on the Houston 2013 reach the maximum when $l = 2$. In summary, due to the flexibility of spatial pooling, it is possible to find the exact layer-number and spatial-size combinations of each dataset by performing repetitive experiments.

TABLE XIV
OA(%) OF SPGCN WITH DIFFERENT TRAINING RATIOS

	2%	5%	8%	10%	15%	20%
Indian	87.57	96.10	98.14	98.47	99.20	99.49
Pavia	99.07	99.61	99.74	99.81	99.85	99.89
Salinas	99.07	99.70	99.81	99.84	99.88	99.92
Houston	91.10	96.36	97.76	98.17	98.93	99.41

F. Impact of Training Ratio

In this section, we explore the performance of the proposed SPGCN with different training ratios. d is set to 6, and the spatial input size is 21×21. Therefore, the spatial size of different layers are set to 21×21, 15×15, 9×9, and 3×3, respectively. We use 2%, 5%, 8%, 10%, 15%, and 20% of labeled samples on the three datasets to train our model. The OAs are summarized in Table XIV. As can be seen, the number of training samples can significantly influence the classification results. The training accuracy increases significantly when the proportion of labeled samples ranges from 2% to 5%, and tends to stabilize when the proportion ranges from 15% to 20%. In addition, the Indian Pines and Houston 2013 datasets generally requires more training samples to reach the same accuracy as the Pavia University and the Salinas.

V. CONCLUSION

Graph convolution can naturally propagate features based on the graph structure. When graphs are properly established, graph convolution can be an effective tool to extract features. In this article, we have proposed a novel SPGCN for HSI classification. To extract robust spectral–spatial features, the model constructs graphs on every local patch and performs graph convolution to extract spectral–spatial features. To refine the feature map obtained by graph convolution, we perform a spatial pooling method based on the property of local regions, which reduces the patch size and the graph size simultaneously. The further analysis illustrates that by harnessing the relationship on graphs, our method effectively fuses features from large square windows to smaller subwindows. Experimental results on three datasets demonstrate that our

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 15:07:25 UTC from IEEE Xplore. Restrictions apply.

GCN-based method achieves competitive performance compared with state-of-the-art CNN-based methods and is fast in speed.

Although the Gaussian similarity enables a short training time, the graph structures may not be representative enough. Therefore, the future direction of our work is to exploit the construction of graphs. On the other hand, it is reasonable to consider that a square subwindow does not always contain the most related pixels. The spatial pooling method can be designed more flexibly, and various pooling methods can be explored to extract more robust features.

REFERENCES

[1] J. M. Bioucas-Dias, A. Plaza, G. Camps-Valls, P. Scheunders, N. M. Nasrabadi, and J. Chanussot, "Hyperspectral remote sensing data analysis and future challenges," *IEEE Geosci. Remote Sens. Mag.*, vol. 1, no. 2, pp. 6–36, Jun. 2013.

[2] M. Teke, H. S. Deveci, O. Haliloglu, S. Z. Gurbuz, and U. Sakarya, "A short survey of hyperspectral remote sensing applications in agriculture," in *Proc. 6th Int. Conf. Recent Adv. Space Technol. (RAST)*, Jun. 2013, pp. 1–5.

[3] M. K. Griffin, M. H. Su, H. H. K. Burke, and J. W. Snow, "Characterization and delineation of plumes, clouds, and fires in hyperspectral images," in *Proc. IEEE Int. Geosci. Remote Sens. Symp.*, Jul. 2000, pp. 809–812.

[4] D. Hong, N. Yokoya, J. Chanussot, and X. X. Zhu, "CoSpace: Common subspace learning from hyperspectral-multispectral correspondences," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 7, pp. 4349–4359, Jul. 2019.

[5] J. P. Ardouin, J. Levesque, and T. A. Rea, "Demonstration of hyperspectral image exploitation for military applications," in *Proc. Int. Conf. Inf. Fusion*, 2007, pp. 1–5.

[6] N. Audebert, B. L. Saux, and S. Lefevre, "Deep learning for classification of hyperspectral data: A comparative review," *IEEE Geosci. Remote Sens. Mag.*, vol. 7, no. 2, pp. 159–173, Jun. 2019.

[7] F. Melgani and L. Bruzzone, "Classification of hyperspectral remote sensing images with support vector machines," *IEEE Trans. Geosci. Remote Sens.*, vol. 42, no. 8, pp. 1778–1790, Aug. 2004.

[8] L. Ma, M. M. Crawford, and J. Tian, "Local manifold learning-based K-nearest-neighbor for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 48, no. 11, pp. 4099–4109, Nov. 2010.

[9] J. Ham, Y. Chen, M. M. Crawford, and J. Ghosh, "Investigation of the random forest framework for classification of hyperspectral data," *IEEE Trans. Geosci. Remote Sens.*, vol. 43, no. 3, pp. 492–501, Mar. 2005.

[10] L. Fang, S. Li, X. Kang, and J. A. Benediktsson, "Spectral–spatial hyperspectral image classification via multiscale adaptive sparse representation," *IEEE Trans. Geosci. Remote Sens.*, vol. 52, no. 12, pp. 7738–7749, Dec. 2014.

[11] G. Licciardi, P. R. Marpu, J. Chanussot, and J. A. Benediktsson, "Linear versus nonlinear PCA for the classification of hyperspectral data based on the extended morphological profiles," *IEEE Geosci. Remote Sens. Lett.*, vol. 9, no. 3, pp. 447–451, May 2012.

[12] T. V. Bandos, L. Bruzzone, and G. Camps-Valls, "Classification of hyperspectral images with regularized linear discriminant analysis," *IEEE Trans. Geosci. Remote Sens.*, vol. 47, no. 3, pp. 862–873, Mar. 2009.

[13] Z. Ye, S. Prasad, W. Li, J. E. Fowler, and M. He, "Classification based on 3-D DWT and decision fusion for hyperspectral image analysis," *IEEE Geosci. Remote Sens. Lett.*, vol. 11, no. 1, pp. 173–177, Jan. 2014.

[14] P. Ghamisi, J. A. Benediktsson, and M. O. Ulfarsson, "Spectral–spatial classification of hyperspectral images based on hidden Markov random fields," *IEEE Trans. Geosci. Remote Sens.*, vol. 52, no. 5, pp. 2565–2574, May 2014.

[15] Y. Chen, Z. Lin, X. Zhao, G. Wang, and Y. Gu, "Deep learning-based classification of hyperspectral data," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 6, pp. 2094–2107, Jun. 2014.

[16] Y. S. Chen, X. Zhao, and X. Jia, "Spectral–spatial classification of hyperspectral data based on deep belief network," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 8, no. 6, pp. 2381–2392, Jun. 2014.

[17] L. Mou, P. Ghamisi, and X. X. Zhu, "Deep recurrent neural networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 7, pp. 3639–3655, Jul. 2017.

[18] Y. Chen, H. Jiang, C. Li, X. Jia, and P. Ghamisi, "Deep feature extraction and classification of hyperspectral images based on convolutional neural networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 10, pp. 6232–6251, Oct. 2016.

[19] W. Song, S. Li, L. Fang, and T. Lu, "Hyperspectral image classification with deep feature fusion network," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 6, pp. 3173–3184, Jun. 2018.

[20] Z. Zhong, J. Li, Z. Luo, and M. Chapman, "Spectral–spatial residual network for hyperspectral image classification: A 3-D deep learning framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 2, pp. 847–858, Feb. 2018.

[21] J. M. Haut, M. E. Paoletti, J. Plaza, A. Plaza, and J. Li, "Visual attention-driven hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 10, pp. 8065–8080, Oct. 2019.

[22] M. Zhu, L. Jiao, F. Liu, S. Yang, and J. Wang, "Residual spectral–spatial attention network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 1, pp. 449–462, Jan. 2021.

[23] X. Mei *et al.*, "Spectral-spatial attention networks for hyperspectral image classification," *Remote Sens.*, vol. 11, no. 8, p. 963, Apr. 2019.

[24] X. Tang, F. B. Mang, and X. R. Zhang, "Hyperspectral image classification based on 3-D octave convolution with spatial–spectral attention network," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 3, pp. 2430–2447, Mar. 2021.

[25] Z. Wang, B. Du, L. Zhang, L. Zhang, and X. Jia, "A novel semisupervised active-learning algorithm for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 6, pp. 3071–3083, Jun. 2017.

[26] X. Zhang, Z. Gao, L. Jiao, and H. Zhou, "Multifeature hyperspectral image classification with local and nonlocal spatial information via Markov random field in semantic space," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 3, pp. 1409–1424, Mar. 2018.

[27] K. Zhu, Y. Chen, P. Ghamisi, X. Jia, and J. A. Benediktsson, "Deep convolutional capsule network for hyperspectral image spectral and spectral-spatial classification," *Remote Sens.*, vol. 11, no. 3, p. 223, Jan. 2019.

[28] X. Zhang, Y. Liang, C. Li, N. Huyan, L. Jiao, and H. Zhou, "Recursive autoencoders-based unsupervised feature learning for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 14, no. 11, pp. 1928–1932, Nov. 2017.

[29] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.

[30] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Proces. Syst. (NIPS)*, 2017, pp. 1024–1034.

[31] W.-L. Chiang, X. Liu, S. Si, Y. Li, S. Bengio, and C.-J. Hsieh, "Cluster-gcn: An efficient algorithm for training deep and large graph convolutional networks," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2019, pp. 257–266.

[32] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional neural networks on graphs with fast localized spectral filtering," in *Proc. Adv. Neural Inf. Proces. Syst. (NIPS)*, 2016, pp. 3844–3852.

[33] Z. Ying, J. You, C. Morris, X. Ren, W. Hamilton, and J. Leskovec, "Hierarchical graph representation learning with differentiable pooling," in *Proc. Adv. Neural Inf. Proces. Syst. (NIPS)*, 2018, pp. 4800–4810.

[34] L. Mou, X. Lu, X. Li, and X. X. Zhu, "Nonlocal graph convolutional networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 12, pp. 8246–8257, Dec. 2020.

[35] J. Bai, B. Ding, Z. Xiao, L. Jiao, H. Chen, and A. C. Regan, "Hyperspectral image classification based on deep attention graph convolutional network," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5504316.

[36] S. Wan, C. Gong, P. Zhong, B. Du, L. Zhang, and J. Yang, "Multiscale dynamic graph convolutional network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 5, pp. 3162–3177, May 2019.

[37] S. Wan, C. Gong, P. Zhong, S. Pan, G. Li, and J. Yang, "Hyperspectral image classification with context-aware dynamic graph convolutional network," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 1, pp. 597–612, May 2020.

[38] Y. Ding, X. Zhao, and Z. Zhang, "Semi-supervised locality preserving dense graph neural network with ARMA filters and context-aware learning for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, early access, Aug. 4, 2021, doi: 10.1109/TGRS.2021.3100578.

[39] S. Wan, S. Pan, and P. Zhong, "Dual interactive graph convolutional networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2021, Art. no. 5510214.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 15:07:25 UTC from IEEE Xplore. Restrictions apply.

[40] D. Hong, L. Gao, J. Yao, B. Zhang, A. Plaza, and J. Chanussot, "Graph convolutional networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 7, pp. 5966–5978, Jul. 2021.
[41] X. He, Y. Chen, and P. Ghamisi, "Dual graph convolutional network for hyperspectral image classification with limited training samples," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2021, Art. no. 5502418.
[42] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. Van Den Berg, I. Titov, and M. Welling, "Modeling relational data with graph convolutional networks," in *Proc. Eur. Semantic Web Conf.* Cham, Switzerland: Springer, 2018, pp. 593–607.
[43] F. Wu, T. Zhang, A. Holanda de Souza, C. Fifty, T. Yu, and K. Q. Weinberger, "Simplifying graph convolutional networks," 2019, *arXiv:1902.07153*.
[44] M. Zhang, Z. Cui, M. Neumann, and Y. Chen, "An end-to-end deep learning architecture for graph classification," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 1–5.
[45] H. Gao and S. Ji, "Graph U-Nets," 2019, *arXiv:1905.05178*.
[46] G. Cucurull, A. Casanova, A. Romero, P. Liü, and Y. Bengio, "Graph attention networks," 2017, *arXiv:1710.10903*.
[47] Y. Zhang, S. Pal, M. Coates, and D. Ustebay, "Bayesian graph convolutional neural networks for semi-supervised classification," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, 2019, pp. 5829–5836.
[48] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
[49] J. Zhu, L. Fang, and P. Ghamisi, "Deformable convolutional neural networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 15, no. 8, pp. 1254–1258, Aug. 2018.



Xiangrong Zhang (Senior Member, IEEE) received the B.S. and M.S. degrees in computer application technology from the School of Computer Science, Xidian University, Xi'an, China, in 1999 and 2003, respectively, and the Ph.D. degree in pattern recognition and intelligent system from the School of Electronic Engineering, Xidian University, in 2006.

From January 2015 to March 2016, she was a Visiting Scientist with the Computer Science and Artificial Intelligence Laboratory, Massachusetts Institute of Technology, Cambridge, MA, USA. She is a Professor with the Key Laboratory of Intelligent Perception and Image Understanding of Ministry of Education, Xidian University. Her research interests include pattern recognition, machine learning, and remote sensing image analysis and understanding.



Shutong Chen received the B.S. degree in intelligent science and technology from Xidian University, Xi'an, China, in 2019, where he is pursuing the M.S. degree with the Key Laboratory of Intelligent Perception and Image Understanding of Ministry of Education.

His research interests include digital image processing and hyperspectral image classification.



Peng Zhu received the B.S. degree from Xidian University, Xi'an, China, in 2017, where he is pursuing the Ph.D. degree.

His research interests include computer vision and remote sensing image analysis.



Xu Tang (Senior Member, IEEE) received the B.Sc., M.Sc., and Ph.D. degrees in electronic circuit and system from Xidian University, Xi'an, China, in 2007, 2010, and 2017, respectively.

From 2015 to 2016, he was a Joint Ph.D. Student along with Prof. W. J. Emery at the University of Colorado at Boulder, Boulder, CO, USA. He is an Associate Professor with the Key Laboratory of Intelligent Perception and Image Understanding of Ministry of Education, Xidian University. He is also a Hong Kong Scholar with Hong Kong Baptist University, Hong Kong. His research interests include remote sensing image content-based retrieval and reranking, hyperspectral image processing, remote sensing scene classification, and object detection.



Jie Feng (Member, IEEE) received the B.S. degree from Chang'an University, Xi'an, China, in 2008, and the Ph.D. degree from Xidian University, Xi'an, in 2014.

She is an Associate Professor with the Key Laboratory of Intelligent Perception and Image Understanding of Ministry of Education, Xidian University. Her research interests include remote sensing image processing, deep learning, and machine learning.



Licheng Jiao (Fellow, IEEE) received the B.S. degree in high voltage from Shanghai Jiao Tong University, Shanghai, China, in 1982, and the M.S. and Ph.D. degrees in electronic engineering from Xi'an Jiaotong University, Xi'an, China, in 1984 and 1990, respectively.

From 1984 to 1986, he was an Assistant Professor with the Civil Aviation Institute of China, Tianjin, China. From 1990 to 1991, he was a Post-Doctoral Fellow with the Key Laboratory for Radar Signal Processing, Xidian University, Xi'an, where he is the Director of the Key Laboratory of Intelligent Perception and Image Understanding of Ministry of Education. He has authored or coauthored more than 200 scientific articles. His research interests include signal and image processing, nonlinear circuits and systems theory, wavelet theory, natural computation, and intelligent information processing.

Dr. Jiao is also a member of the IEEE Xi'an Section Executive Committee and an Executive Committee Member of the Chinese Association of Artificial Intelligence. He is the Chairperson of the Awards and Recognition Committee.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 15:07:25 UTC from IEEE Xplore. Restrictions apply.